

Designing an Intelligent Reactive Multi-Domain Polymarket Trading Agent

Yiwei LI
School of Computer Science
University of Nottingham
Ningbo, China
20513831
scyy130@nottingham.edu.cn

Member B
School of Computer Science
University of Nottingham
Ningbo, China

Member C
School of Computer Science
University of Nottingham
Ningbo, China

Member D
School of Computer Science
University of Nottingham
Ningbo, China

Abstract—This study presents a reactive autonomous trading agent for multi-domain prediction markets on Polymarket. The agent treats Polymarket as an environment containing three sub-environments: Counter-Strike match markets, Bitcoin 5-minute up/down markets, and weather event markets. At each decision point, a data hub produces structured domain features, a news hub uses a large language model to score current textual evidence, and the agent compares its estimated probability with the market-implied probability to estimate market edge. These components are fused into a single opportunity score for each domain. The agent’s action rule is reactive: it selects the highest-scoring domain opportunity for a signal or fixed-stake simulated trade, maintains that domain while it remains the best opportunity, switches domain when another domain receives a higher opportunity score, exits when the best opportunity score falls below threshold, or holds when no score passes the threshold. The evaluation compares the proposed fusion and selection mechanism with market-only, data-only, news-only, and simpler fusion baselines across the three domains.

Index Terms—reactive agents, Polymarket, prediction markets, multi-domain agents, opportunity scoring, simulated trading

I. INTRODUCTION

Prediction markets provide a suitable environment for studying autonomous decision-making agents because they expose uncertain future events as dynamic prices and aggregate dispersed information into probabilistic forecasts [1]–[3]. On Polymarket, binary outcome prices are explicitly presented as implied probabilities, while market data and order-book information are available through public APIs [4], [5]. A market price, however, does not necessarily capture all domain-specific data or recent public information [6], [7]. This study therefore asks whether an intelligent agent can react to multiple Polymarket domains and select the highest-valued current opportunity.

The system is framed as a reactive multi-domain trading agent. The overall environment is the Polymarket opportunity space, and the agent can observe three sub-environments: Counter-Strike match markets, Bitcoin 5-minute up/down mar-

kets, and weather event markets. Each domain reports a current opportunity estimate based on structured data features and news-derived evidence. The agent does not attempt to plan a long sequence of trades. Instead, at each decision point it compares the three current opportunity scores and selects only the single highest-scoring opportunity, or holds when no domain passes a decision threshold.

The multi-domain setting matters because a single-domain predictor cannot allocate action across unrelated markets. A weather market can contain a forecast-driven edge while Bitcoin markets have no clear direction, or a Counter-Strike roster update can create a short-lived match-level edge. The agent is therefore evaluated as an opportunity allocator, not only as a probability estimator. Its intelligence is expressed through switching: the agent can hold when no domain reaches the opportunity threshold, enter the domain with the highest current certainty-adjusted opportunity, maintain that domain while it remains dominant, switch to another domain when the evidence changes, and exit when the active opportunity falls below threshold. This framing matches the coursework focus on an autonomous agent in an environment: the environment reports changing conditions, the agent chooses one action from multiple possible sub-environments, and the evaluation measures whether that choice improves over simpler reactive rules.

This design follows the standard agent model in which an agent perceives an environment and selects actions from those perceptions [8]. The agent perceives its environment through domain reports, processes those reports through a fixed opportunity scoring rule, and acts by producing one probability/confidence signal and a fixed-stake simulated trading decision. The action space includes hold, enter, maintain, switch, and exit decisions. The system is reactive because the action is determined by the current percepts rather than by an explicit long-horizon plan, following the behaviour-based tradition associated with reactive control [9].

The central research question is:

Does a reactive multi-domain agent that fuses structured data, large language model news evidence, and market-edge estimation produce higher-quality Polymarket opportunity selection than market-only or single-source baselines?

Opportunity-selection quality is defined operationally by hit rate, Brier score, simulated profit/loss, return per selected signal, and coverage. A method is therefore not judged only by whether it predicts likely outcomes, but by whether its acted signals are more accurate, better calibrated, or more profitable at the reported level of selectivity.

This question is broken into four experimental questions:

- 1) Does combining data-side and news-side evidence improve signal quality compared with either source alone?
- 2) Does comparing each estimated probability with the Polymarket-implied probability improve opportunity selection?
- 3) Does one unified reactive selector behave differently across Counter-Strike, Bitcoin, and weather markets?
- 4) How sensitive is the selector to the decision threshold τ ?

The contribution of this study is a single agent architecture rather than three unrelated predictors. The study defines the multi-domain environment, the data hub and news hub, the opportunity scoring mechanism, the reactive action rule, and the experimental methodology used to compare the proposed agent against simpler baselines.

II. BACKGROUND AND RELATED WORK

A. Reactive Intelligent Agents

An intelligent agent can be described as a system that perceives an environment, uses a decision procedure to process those perceptions, and selects actions that affect the environment or the next decision cycle [8]. In the simplest case, a reactive agent maps current inputs directly to actions through thresholds, conditions, or scoring rules. This is less expressive than a planning agent, but it fits environments that change quickly and require the highest-scoring action from the current state. Brooks' subsumption architecture demonstrates how layered reactive behaviour can produce stable autonomous control without relying on a single symbolic world model [9].

The agent in this project is reactive in this sense. It observes the latest reports from the three Polymarket domains, computes opportunity scores, and acts on the highest-scoring opportunity. The intelligent component is not a long-term planner, but the design of the scoring function and the integration of structured data, news evidence, and market edge.

B. Prediction Markets and Market-Implied Probability

Prediction markets trade contracts whose payoff depends on a future event. Prior work presents these markets as mechanisms for aggregating distributed beliefs and producing probabilistic forecasts [1], [3]. Market design also matters because the contract structure, liquidity, and payoff rule shape

how information is revealed [2]. In a binary market, the price of a YES contract can be interpreted as an implied probability after accounting for market frictions. Polymarket follows this convention by pricing shares between zero and one dollar, with the price directly representing the market's belief in the outcome probability [5]. Polymarket markets are event contracts: a YES share pays out if the stated event occurs, a NO share pays out if it does not, and market resolution converts the contract into a binary outcome for evaluation. Multi-outcome markets can be handled as sets of mutually exclusive binary outcomes. However, a high probability is not automatically a good opportunity. If the market price already matches the agent's probability estimate, the expected edge may be small [6], [7].

This distinction is central to the paper. The agent does not only look for certainty; it looks for opportunity. Opportunity requires both evidence about the likely outcome and a difference between the agent's estimate and the current market-implied probability.

C. Financial Signal Agents

Financial agents perceive market state and issue actions such as buy, sell, or hold. A purely reactive financial agent may respond to the current price, spread, or time-to-close, reflecting the importance of liquidity and order-book state in electronic markets [10], [11]. A stateful agent may also retain historical information, such as recent transactions or calibration errors. In this study, the emphasis is on reactive selection across domains, but the agent still uses domain reports that may include features computed from historical data.

The proposed system is evaluated through fixed-stake simulated trading rather than live execution. This keeps the coursework focused on agent design and experimental analysis, while avoiding claims about real trading performance under liquidity, slippage, and execution-delay constraints [10], [11].

D. News Evidence and Large Language Model Scoring

Prediction-market events are often affected by public textual information such as news, reports, team updates, weather warnings, or market commentary. Financial research has shown that media tone and textual features can contain market relevant information [12], [13]. This study represents the text side through a news hub. The news hub collects raw text from sources such as RSS feeds or other news inputs and uses a large language model to produce a normalized news score for each domain report. Large language models are suitable for this scoring role because they can perform task adaptation from natural-language instructions and examples [14], [15].

The role of the large language model is limited and explicit: it converts unstructured text into a score that can be fused with structured data features. The final action is still selected by the reactive agent rule, not by an open-ended conversational agent.

E. Evaluation of Agent Behaviour

The module emphasises that agent systems are evaluated through structured experiments. For this study, the main evaluation is comparative: the proposed multi-domain selector is

compared with market-only, data-only, news-only, and simpler fusion baselines. The results report both prediction quality and simulated opportunity-selection performance. Suitable metrics include accuracy, Brier score, hit rate, simulated profit and loss, return per selected signal, coverage, and sensitivity to the decision threshold. The Brier score is a standard metric for probabilistic forecasts [16], and proper scoring rules provide a principled framework for evaluating probability estimates [17].

III. SYSTEM DESIGN

A. Environment and Sub-Environments

The agent’s environment is the Polymarket opportunity space. Polymarket’s Gamma API provides market discovery and metadata, while the CLOB API provides prices, order books, spread, midpoint, and historical price endpoints [4], [18]. The environment is split into three sub-environments:

- Counter-Strike match markets, where the target is the outcome of an esports match or event.
- Bitcoin 5-minute up/down markets, where the target is the short-term direction of the Bitcoin price.
- Weather event markets, where the target is the outcome of a weather condition or threshold event.

At a decision time t , each sub-environment produces a domain report. A report contains the relevant Polymarket market state, domain data features, news-derived evidence, and any already computed probability estimates.

B. Domain Characteristics

The three domains differ in horizon, evidence type, and source reliability. Counter-Strike markets are event-level esports markets. Their evidence is discrete and contextual: team form, player availability, roster changes, map pool, tournament format, and recent match outcomes can all affect the match probability [19], [20]. These markets are sensitive to late roster or travel information, so news evidence can change the opportunity score quickly.

Bitcoin 5-minute markets are short-horizon financial markets. Their target is a directional move over a narrow time window, so recent price movement, volatility, spread, and macro or crypto news must be interpreted at high frequency. A signal can become stale within minutes, and the market-implied probability may already reflect public price momentum [11], [21], [22].

Weather markets are physical event markets. Their evidence is comparatively structured because numerical forecasts can be mapped to temperature buckets or event thresholds [17], [23]. News still matters when it reports cold fronts, rain, wind, or other local conditions that shift the forecast. Weather therefore provides the clearest example of combining structured forecast data with textual adjustment evidence.

C. Agent Loop and Report Interface

The agent loop has four stages. First, the environment layer retrieves the current Polymarket state and the latest domain inputs. Second, the data hub and news hub convert

heterogeneous inputs into normalized report fields. Third, the agent computes probability, edge, and opportunity scores for the three domains. Fourth, the action selector emits a hold, enter, maintain, switch, or exit decision. This loop is repeated at each decision time without retaining a long-horizon plan.

The report interface is intentionally narrow. Every domain report contains a market identifier, a timestamp, the market-implied probability, a data-side score, a news-side score, and a candidate probability estimate. Domain-specific features remain inside the data hub, while the selector only sees comparable summary fields. This separation keeps the system modular: Counter-Strike, Bitcoin, and weather feature engineering can change without changing the reactive action rule.

D. Data Hub

The data hub is responsible for structured domain features. These features are different across domains, but they are converted into a common normalized data-side score $D_d(t)$ for each domain d .

For Counter-Strike markets, data-side features include market price, match-level information, team-related signals, and historical outcome features. Esports analytics studies show that Counter-Strike telemetry and structured game-state features can support short-horizon predictive tasks [19], [20]. For Bitcoin markets, features include recent price movement, short-window technical indicators, volatility, and time-to-resolution. Prior work on intraday Bitcoin forecasting supports the use of machine learning and technical features for high-frequency cryptocurrency prediction [21], [22]. For weather markets, features include weather forecasts, event thresholds, time remaining, and historical forecast or outcome information [23]. The implementation varies by domain, but the output interface is the same: one data-side score per domain.

The data hub also protects the backtest from information leakage. Each feature is timestamped and only information available before the decision time is included in the report. Market prices are read as the entry state for the decision rather than as post-resolution evidence. When a feature is missing, the domain report records the missing field explicitly and the data-side score is computed from the available inputs using the same normalization rule. This is necessary because the three domains do not provide equally dense historical records [24].

E. News Hub

The news hub handles unstructured text. It receives RSS entries, news snippets, or other raw textual inputs relevant to each domain. A large language model then scores whether the news evidence supports a higher or lower probability for the market outcome. The output is a normalized news-side score $N_d(t)$.

Using a large language model here avoids hard-coding separate keyword rules for all domains. The model is not the whole agent; it is one scoring component whose output is fused with structured data and market-edge information [14], [15].

The news prompt is constrained to return a structured score rather than open ended advice. For each market, the model receives the market question, the domain label, the relevant text snippets, and the current market direction under consideration. Its output is interpreted as evidence strength, not as a final trading decision. The score is normalized onto the same scale as the data-side score, and uncertain or irrelevant text is mapped close to neutral. This keeps the news hub as an input to reactive scoring while final control remains inside the agent architecture.

F. Market Edge

For each domain, the agent compares its estimated probability $\hat{p}_d(t)$ with the Polymarket-implied probability $p_d^{mkt}(t)$. The edge component is:

$$E_d(t) = |\hat{p}_d(t) - p_d^{mkt}(t)|. \quad (1)$$

The sign of $\hat{p}_d(t) - p_d^{mkt}(t)$ determines the direction of the signal, while the magnitude contributes to the opportunity score. This prevents the agent from treating high-certainty but fairly priced markets as actionable opportunities [6], [7].

G. Opportunity Score

The final opportunity score combines the data-side score, the news-side score, and the market-edge score. The unified scoring rule used throughout the paper is:

$$O_d(t) = D_d(t) + N_d(t) + E_d(t). \quad (2)$$

All three components are normalized to $[0, 1]$ before fusion, so $O_d(t) \in [0, 3]$. The same formula is used for all domains so that the selector can compare opportunities directly. Domain-specific calibration is treated as a limitation and extension rather than built into the primary design, because the main experimental question concerns a single comparable selector over heterogeneous markets.

The formula is simple by design. The coursework goal is to evaluate the agent architecture and the value of cross-domain reactive selection, so the primary score avoids a learned meta-model whose behaviour would be difficult to interpret from a small number of backtests. The additive score also makes ablation baselines clear: removing $D_d(t)$, $N_d(t)$, or $E_d(t)$ directly tests the contribution of structured data, news evidence, and market edge.

H. Reactive Action Selection and Switching

The agent has one active simulated position at most. Let $c(t)$ denote the currently active domain, where $c(t) = \emptyset$ means that the agent is not currently trading. At each decision time, it first selects the domain with the highest opportunity score:

$$d^*(t) = \operatorname{argmax}_{d \in \mathcal{D}} O_d(t), \quad (3)$$

where $\mathcal{D}$ contains the three domains. The action rule is:

- if $O_{d^*}(t) < \tau$ and $c(t) = \emptyset$, hold;
- if $O_{d^*}(t) < \tau$ and $c(t) \neq \emptyset$, exit the active simulated position;

- if $O_{d^*}(t) \geq \tau$ and $c(t) = \emptyset$, enter domain $d^*(t)$;
- if $O_{d^*}(t) \geq \tau$ and $c(t) = d^*(t)$, maintain the current domain;
- if $O_{d^*}(t) \geq \tau$ and $c(t) \neq d^*(t)$, switch by exiting the current domain and entering $d^*(t)$.

Switching is the core reason for using multiple domains. The agent does not rank Counter-Strike, Bitcoin, and weather as separate projects; it treats them as competing sub-environments and allocates its single active simulated trade to the domain with the highest current certainty-adjusted opportunity.

The threshold uses the same scale as $O_d(t)$. The operating value of τ is selected before final testing from the grid $\{0.25, 0.50, \dots, 2.75\}$ by maximizing validation return per selected signal subject to a minimum validation coverage of 10%. The final backtest then keeps this value fixed. The threshold-sensitivity experiment reports the same metrics across the full grid to show how selectivity changes the results [24].

This design makes the agent selective. It does not trade every domain whenever a prediction is available. It enters only when a domain passes the threshold, switches when another domain becomes the highest-scoring opportunity, and exits when no domain remains above the opportunity threshold.

IV. IMPLEMENTATION TECHNOLOGIES AND CHALLENGES

A. Implementation Stack

The implementation is organised as a Python signal and backtesting pipeline. A command-line runner executes the domain modules, loads market configuration, and writes structured output files. Market definitions, domain settings, news sources, and historical decision windows are stored in configuration files so that experiments can be repeated without editing the code for each run.

The system uses Polymarket market metadata, price, and order-book interfaces as the market layer [4], [5]. Domain evidence is collected from public application programming interfaces and web feeds. Weather features use forecast data and weather news; Bitcoin features use market-window indicators and cryptocurrency, macroeconomic, and regulatory news; Counter-Strike features use match, team, roster, and event-related evidence. Tabular data processing is used to build the experimental tables, while JSONL and CSV outputs preserve both machine-readable evidence and human-readable inspection files.

B. Reproducibility Protocol

The submitted code reproduces the reported experiment from configuration and logged data rather than from manual notebook edits. A reproduction run has three inputs: the market configuration file, the resolved-outcome file, and the experiment configuration containing the decision windows, threshold grid, and baseline list [25]. The command-line entry point accepts the selected domains, the configuration path, the output directory, and optional start, end, and as-of timestamps for historical runs.

For each decision time, the runner writes the domain reports, selected signal, top evidence, and metric row. The result tables in this report are produced by aggregating those output files. Reproducing a table therefore requires the same configuration, the same resolved outcomes, and the same random seed for any stochastic large language model scoring calls. When deterministic model settings are used, the same inputs produce the same domain reports and selected actions.

C. Evidence and Signal Outputs

Each run produces three main artefacts. The evidence log records the raw or semi-processed inputs used by the agent, including source, title, timestamp, summary, event tags, relevance, and source weight. The signal log records the domain, target market, direction score, confidence, estimated probability shift, top evidence, and domain-specific metadata. The results table aggregates these signals into the metrics used in the evaluation. This structure makes the backtest auditable: a selected action can be traced from final metric, to signal record, to evidence item.

This output design also supports the coursework requirement for experimental evaluation. Repeated runs with different decision windows or thresholds produce comparable output files, and the same analysis script can aggregate each condition into hit rate, Brier score, profit/loss, and coverage. The code therefore treats experiments as parameterised runs rather than as manual one-off demonstrations.

D. Implementation Challenges

The first implementation challenge is heterogeneity. The three domains have different data shapes: weather produces multi-outcome temperature buckets, Bitcoin produces short-window directional decisions, and Counter-Strike produces match-level signals from team and event evidence. The report interface resolves this by converting every domain into comparable fields before action selection.

The second challenge is temporal validity. A backtest is only meaningful if the agent uses information available at the decision time. The implementation therefore attaches timestamps to evidence and supports start, end, and as-of windows. This prevents later news or post-resolution information from leaking into a historical decision [24].

The third challenge is noisy textual evidence. News feeds can contain duplicate items, irrelevant stories, delayed timestamps, and source outages. The news hub therefore filters by relevance, keeps source metadata, records source failures, and maps uncertain evidence toward neutral scores. Large language model scoring is constrained to a fixed numerical output so that it acts as a scoring tool rather than as an uncontrolled final decision-maker [14], [15].

The final challenge is comparability across domains. A weather forecast score, a Bitcoin technical score, and a Counter-Strike roster score are not naturally on the same scale. The implementation normalizes the component scores before fusion and uses market edge to align all domains around the

same question: whether the agent’s estimated probability differs enough from the current Polymarket-implied probability to justify action.

V. EXPERIMENTAL METHODOLOGY

A. Research Questions

The experiments are designed to answer the following questions:

- 1) Does the proposed opportunity score improve over market-implied probability alone?
- 2) Does fusing structured data and large language model news scoring improve over data-only or news-only scoring?
- 3) Does the reactive cross-domain selector choose higher-quality opportunities than acting independently in every domain?
- 4) How does performance differ across Counter-Strike, Bitcoin 5-minute, and weather markets?

B. Experimental Conditions

The proposed agent is compared with the following baselines:

- Market-only baseline: uses Polymarket-implied probability without external data or news scoring.
- Data-only baseline: uses the data hub score $D_d(t)$ without the news hub.
- News-only baseline: uses the large language model news score $N_d(t)$ without structured data features.
- Fusion baseline: combines data and news evidence but does not include market edge.
- Proposed agent: combines data score, news score, and market edge, then reacts by holding, entering, maintaining, switching, or exiting according to the highest-scoring domain opportunity.

C. Parameter and Threshold Protocol

All methods use the same historical decision windows and resolved outcomes. Raw data and news features are transformed into the normalized scores $D_d(t)$ and $N_d(t)$, and market edge is computed as $E_d(t)$. The proposed agent then uses $O_d(t) = D_d(t) + N_d(t) + E_d(t)$. The threshold τ is selected on a validation split from the pre-specified grid $\{0.25, 0.50, \dots, 2.75\}$. The selected τ is then frozen for the final test comparison. The selection criterion is validation return per selected signal, subject to a minimum validation coverage of 10%. The threshold-sensitivity analysis repeats the test evaluation over the full grid and reports the effect on coverage and signal quality [24].

D. Backtesting Procedure

For each historical decision time, the system constructs domain reports for the three sub-environments. Each method produces a probability estimate or signal. For the proposed agent, the selector computes $O_d(t)$ for each domain and compares the maximum score with threshold τ . If no active simulated position exists, the agent either holds or enters the

highest-scoring domain. If an active simulated position exists, the agent maintains it, switches to a different domain, or exits depending on the new domain scores. Entered positions that remain open to resolution are evaluated from the resolved market outcome; positions exited or switched before resolution are marked to market using the available market-implied probability at the exit decision.

Decision windows are constructed before the outcome is observed. For binary markets, the resolved YES outcome is encoded as $y = 1$ and the resolved NO outcome as $y = 0$. The market-implied probability is read from the available price at the decision time, while the agent probability is computed only from features and news available at or before that time. This preserves the causal ordering required for a valid backtest [24].

The backtest is repeated across the markets and decision windows defined in the experiment configuration. Where the scoring process contains randomness, the experiment uses multiple runs and reports averages and variation.

E. Metrics

The evaluation uses both prediction metrics and opportunity-selection metrics:

- Accuracy or hit rate: the proportion of selected signals whose direction was correct.
- Brier score: squared error between predicted probability and outcome [16], [17].
- Simulated profit and loss: fixed-stake return under the selected signal.
- Return per selected signal: simulated profit divided by the number of selected opportunities.
- Coverage: the number or proportion of decision times where the method acted instead of holding.
- Switch and exit counts: the number of times the proposed agent changed active domain or left the market because no domain met threshold.
- Threshold sensitivity: how performance changes as τ changes.

Simulated profit and loss uses a fixed stake for every acted signal. If the agent selects the YES side, the entry price is the market-implied probability for YES; if it selects the NO side, the entry price is the complementary probability. A correct fixed-stake signal receives the contract payoff net of entry cost, while an incorrect signal loses the entry cost. This abstraction keeps the comparison focused on selection quality rather than position sizing.

The Brier score for a binary outcome is $(\hat{p}_{YES} - y)^2$, where $\hat{p}_{YES}$ is the agent's probability for the YES outcome and $y \in \{0, 1\}$ is the resolved YES outcome. Coverage is the number of acted decision windows divided by the total number of available decision windows. Return per selected signal is total simulated profit/loss divided by the number of acted signals, so it separates selectivity from total activity. Switch and exit counts are reported to show whether the multi-domain agent is actively reacting to changing opportunities or simply staying in one domain.

F. Analysis Plan

Results are presented at two levels. First, per-domain results show how the agent behaves in Counter-Strike, Bitcoin, and weather markets. Second, cross-domain selection results quantify the effect of choosing the single highest-scoring opportunity instead of acting on every domain independently. Switch and exit analysis then shows whether the selector used its action space to reallocate exposure as opportunity scores changed.

The analysis uses tables, plots, descriptive statistics, and concise comparisons against the research questions. Where the final sample contains enough independent decision windows, comparative tests are reported alongside the descriptive statistics; otherwise, the analysis relies on confidence intervals, effect sizes, and direct baseline comparisons [26].

VI. EXPERIMENTAL RESULTS

The completed backtest evaluates the agent as a sequence of time-indexed selection decisions. Each method observes the same resolved Polymarket markets and the same historical decision windows. The market-only, data-only, news-only, and data-plus-news baselines isolate the contribution of each information source, while the proposed agent tests whether the reactive cross-domain selector improves the quality of the acted-upon opportunities.

A. Overall Baseline Comparison

Table I reports aggregate performance across all three domains. The comparison uses both predictive quality and opportunity selection quality: hit rate measures whether selected directions were correct, Brier score measures probabilistic accuracy, simulated profit/loss measures the fixed-stake trading outcome, and coverage records how often each method chose to act rather than hold.

TABLE I
OVERALL BASELINE COMPARISON ACROSS ALL DOMAINS.

Method	Hit rate	Brier	P/L	Signals	Coverage
Market-only	[P]	[P]	[P]	[P]	[P]
Data-only	[P]	[P]	[P]	[P]	[P]
News-only	[P]	[P]	[P]	[P]	[P]
Data + news	[P]	[P]	[P]	[P]	[P]
Proposed agent	[P]	[P]	[P]	[P]	[P]

The proposed agent selected [P] opportunities from [P] available decision windows, giving coverage of [P]%. Its hit rate was [P]%, its Brier score was [P], and its simulated profit/loss was [P]. Relative to the market-only baseline, this corresponds to a hit-rate change of [P] percentage points, a Brier-score change of [P], and a profit/loss change of [P]. Relative to the data-plus-news baseline, the market-edge term changed simulated profit/loss by [P] and changed coverage by [P] percentage points. These differences separate changes in opportunity-selection quality from changes in the number of acted-upon markets.

The proposed agent made [P] domain switches and [P] exits. These counts indicate how often the multi-domain design

changed the active market instead of remaining in the first selected domain.

B. Per-Domain Results

Table II separates the proposed agent’s performance by sub-environment. This breakdown is needed because the three domains expose different information structures: Counter-Strike markets depend on team and match context, Bitcoin 5-minute markets are dominated by short-horizon price movement, and weather markets are linked to forecast evidence [19]–[21], [23]. The same unified formula is therefore tested as a cross-domain mechanism rather than as a domain-specific predictor.

TABLE II
PER-DOMAIN PERFORMANCE OF THE PROPOSED AGENT.

Domain	Hit rate	Brier	P/L	Signals	Share
Counter-Strike	[P]	[P]	[P]	[P]	[P]
Bitcoin 5-minute	[P]	[P]	[P]	[P]	[P]
Weather	[P]	[P]	[P]	[P]	[P]

The highest-profit domain was [P], with a hit rate of [P]% and profit/loss of [P]. The weakest domain by the same measure was [P], with a hit rate of [P]% and profit/loss of [P]. The selector allocated [P]% of its acted decisions to Counter-Strike, [P]% to Bitcoin 5-minute markets, and [P]% to weather markets. This distribution shows the degree to which the agent concentrated on one consistently profitable domain versus reacting to high-scoring opportunities across all three sub-environments.

C. Threshold Sensitivity

The threshold sensitivity experiment varies τ , the minimum opportunity score required before the agent acts. Lower thresholds increase coverage but allow weaker signals into the selected set; higher thresholds make the agent more selective but reduce the number of evaluated opportunities. Table III reports the resulting tradeoff.

TABLE III
SENSITIVITY OF THE PROPOSED AGENT TO THE DECISION THRESHOLD.

τ	Hit rate	Brier	P/L	Coverage
[P]	[P]	[P]	[P]	[P]
[P]	[P]	[P]	[P]	[P]
[P]	[P]	[P]	[P]	[P]
[P]	[P]	[P]	[P]	[P]

The reported operating threshold is $\tau = [P]$. At this setting, the agent achieved a hit rate of [P]%, a Brier score of [P], and coverage of [P]%. Reducing the threshold to $\tau = [P]$ changed coverage by [P] percentage points and changed profit/loss by [P]. Increasing the threshold to $\tau = [P]$ changed coverage by [P] percentage points and changed profit/loss by [P]. The sensitivity analysis therefore indicates how much the agent’s performance depends on stable opportunity ranking rather than one narrow threshold choice.

D. Representative Signals

Table IV gives representative acted decisions from the backtest. Each row records the current domain, the selected market, the market-implied probability, the agent’s estimated probability, the edge, the opportunity score, the selected action, and the realised outcome. The examples include both successful and unsuccessful signals so that the qualitative interpretation does not depend on success cases alone.

TABLE IV
REPRESENTATIVE SIGNALS SELECTED BY THE REACTIVE CROSS-DOMAIN AGENT.

Domain	Market	p_m	$\hat{p}$	Edge	Score	Action	Outcome
Counter-Strike	[P]	[P]	[P]	[P]	[P]	[P]	[P]
Bitcoin 5-minute	[P]	[P]	[P]	[P]	[P]	[P]	[P]
Weather	[P]	[P]	[P]	[P]	[P]	[P]	[P]
Switch or exit case	[P]	[P]	[P]	[P]	[P]	[P]	[P]

The successful cases show how the selector combines external evidence and market edge when Polymarket prices leave exploitable disagreement. The switch or exit case shows how the agent reacts when the highest-scoring opportunity changes or falls below threshold. This makes the result analysis auditable rather than relying only on aggregate metrics.

VII. DISCUSSION

A. Interpreting the Reactive Selector

The proposed agent is intentionally reactive. Its behaviour is determined by the current reports from the three domains rather than by a long-term trading plan. This matches the coursework question because the goal is to evaluate whether the agent can identify the highest-scoring current opportunity across heterogeneous Polymarket sub-environments.

The key design choice is to score opportunity rather than certainty. A market can have a high probability of resolving YES but still be unattractive if the Polymarket price already reflects that likelihood. By including market edge, the agent is designed to prefer cases where its estimate differs meaningfully from the market-implied probability [6], [7].

This choice also clarifies why the system is an agent rather than a collection of prediction scripts. The data and news components generate percepts, but the selector must still decide which sub-environment deserves the single action at the current time. The multi-domain design makes that decision non-trivial: the agent can stay out of the market, enter one domain, maintain that domain, switch to another domain, or exit when the best opportunity score falls below threshold. These actions make the system more agent-like than three independent predictors because the selector controls both market choice and timing.

The table-level reporting also makes the decision trace visible, connecting the quantitative results to the perception-action loop required by the module.

B. Domain Differences

The three domains behaved differently in the backtest. Bitcoin 5-minute markets were short-horizon and noisy, so apparent opportunities were sensitive to brief price movements. Weather markets were more directly connected to structured external evidence because forecasts could be mapped to event probabilities. Counter-Strike markets depended more heavily on team context, match format, and the quality of available news or roster information [19]–[21], [23].

The highest-profit domain was [P], while the lowest-profit domain was [P]. This supports a domain-level interpretation rather than a purely aggregate one. A positive aggregate result is meaningful only if it is not driven by one domain masking systematic failure in another; conversely, a weak aggregate result can still identify a sub-environment where the reactive design is effective.

C. Limitations

The first limitation is data availability. Historical Polymarket prices, resolved outcomes, news availability, and domain features may not be equally complete across all three domains. The second limitation is that the backtest uses fixed-stake simulated trading rather than live execution, so it does not fully capture liquidity, slippage, fees, or execution delay [10], [11]. The third limitation is that large language model news scores may be noisy or inconsistent unless the prompt and scoring scale are carefully controlled [15].

The scoring rule is also simple. A unified formula makes the cross-domain comparison clear, but it may not be optimal for every domain. Further calibration, domain-specific weights, or liquidity-aware risk adjustment are possible extensions.

D. Connection to Intelligent Agent Design

The system demonstrates the module concepts of perception, reactive action selection, and experimental evaluation. The data hub and news hub provide percepts; the opportunity score is the reactive action-selection mechanism; and the hold, enter, maintain, switch, and exit decisions are the agent actions. The evaluation then tests whether this design improves performance compared with simpler baselines.

VIII. AUTHOR CONTRIBUTIONS

All four members contributed equally to the project. Yiwei LI was responsible for the overall system design and the Bitcoin 5-minute market component. Member B was responsible for the data hub and the Counter-Strike market prediction component. Member C was responsible for the news hub and large language model news scoring component. Member D was responsible for the backtesting system and the weather market component.

IX. CONCLUSION

This study presents a reactive multi-domain Polymarket trading agent. The agent observes three sub-environments, Counter-Strike match markets, Bitcoin 5-minute up/down markets, and weather event markets, then decides whether to

hold, enter, maintain, switch, or exit based on the highest-scoring current opportunity. The design combines structured data features, large language model news scoring, and market-edge estimation into a unified opportunity score.

The evaluation shows that the central issue is not only whether each domain can produce a reasonable probability estimate, but whether an agent can use those estimates to choose the most actionable market at a given time. The strongest observed result was in [P], where the selector most clearly converted domain evidence and market edge into useful acted signals. Across the remaining domains, the results show the tradeoff between selectivity, coverage, and cross-domain switching, which is discussed in detail in the preceding section. Overall, the study supports evaluating the system as a single reactive selector rather than as three independent domain-specific predictors.

Future work includes improved probability calibration, more reliable news scoring prompts, live paper-trading evaluation, liquidity-aware scoring, and domain-specific extensions if the unified formula proves too simple for one of the three markets.

REFERENCES

- [1] J. Wolfers and E. Zitzewitz, “Prediction markets,” *Journal of Economic Perspectives*, vol. 18, no. 2, pp. 107–126, 2004.
- [2] Y. Chen and D. M. Pennock, “Designing markets for prediction,” *AI Magazine*, vol. 31, no. 4, pp. 42–52, 2010.
- [3] K. J. Arrow *et al.*, “The promise of prediction markets,” *Science*, vol. 320, no. 5878, pp. 877–878, 2008.
- [4] Polymarket, “Gamma api: Events, markets, and discovery,” <https://docs.polymarket.com/developers/gamma-markets-api/overview>, 2026, accessed: 2026-05-01.
- [5] —, “Prices and orderbook,” <https://docs.polymarket.com/polymarket-learn/trading/using-the-orderbook>, 2026, accessed: 2026-05-01.
- [6] E. F. Fama, “Efficient capital markets: A review of theory and empirical work,” *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970.
- [7] S. J. Grossman and J. E. Stiglitz, “On the impossibility of informationally efficient markets,” *The American Economic Review*, vol. 70, no. 3, pp. 393–408, 1980.
- [8] M. Wooldridge and N. R. Jennings, “Intelligent agents: Theory and practice,” *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.
- [9] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [10] T. Hendershott, C. M. Jones, and A. J. Menkveld, “Does algorithmic trading improve liquidity?” *The Journal of Finance*, vol. 66, no. 1, pp. 1–33, 2011.
- [11] R. Cont, S. Stoikov, and R. Talreja, “A stochastic model for order book dynamics,” *Operations Research*, vol. 58, no. 3, pp. 549–563, 2010.
- [12] P. C. Tetlock, “Giving content to investor sentiment: The role of media in the stock market,” *The Journal of Finance*, vol. 62, no. 3, pp. 1139–1168, 2007.
- [13] T. Loughran and B. McDonald, “When is a liability not a liability? textual analysis, dictionaries, and 10-ks,” *The Journal of Finance*, vol. 66, no. 1, pp. 35–65, 2011.
- [14] T. B. Brown *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [15] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, “Calibrate before use: Improving few-shot performance of language models,” in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139. PMLR, 2021, pp. 12 697–12 706.
- [16] G. W. Brier, “Verification of forecasts expressed in terms of probability,” *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

- [17] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.
- [18] Polymarket, "Overview of the polymarket apis," <https://docs.polymarket.com/api-reference>, 2026, accessed: 2026-05-01.
- [19] R. Herbrich, T. Minka, and T. Graepel, "TrueSkill: A bayesian skill rating system," in *Advances in Neural Information Processing Systems*, vol. 19, 2007, pp. 569–576.
- [20] S. Marshall, P. M. Blom, and P. Spronck, "Enabling real-time prediction of in-game deaths through telemetry in counter-strike: Global offensive," in *Proceedings of the 17th International Conference on the Foundations of Digital Games*, 2022, pp. 1–11.
- [21] Y. Liu and A. Tsyvinski, "Risks and returns of cryptocurrency," *The Review of Financial Studies*, vol. 34, no. 6, pp. 2689–2727, 2021.
- [22] S. Lahmiri and S. D. Bekiros, "Intelligent forecasting with machine learning trading systems in chaotic intraday bitcoin market," *Chaos, Solitons & Fractals*, vol. 133, p. 109641, 2020.
- [23] P. Bauer, A. Thorpe, and G. Brunet, "The quiet revolution of numerical weather prediction," *Nature*, vol. 525, no. 7567, pp. 47–55, 2015.
- [24] H. White, "A reality check for data snooping," *Econometrica*, vol. 68, no. 5, pp. 1097–1126, 2000.
- [25] R. D. Peng, "Reproducible research in computational science," *Science*, vol. 334, no. 6060, pp. 1226–1227, 2011.
- [26] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.